ESTR1002 Fall 2025

Lab Exercise #9

Due: 26 Nov 2025

Separate your answer for each question in a different .c file. Compress all files into a .zip and submit it to Blackboard.

Question 1

We want to store a mini dictionary. When we print out the whole dictionary, we can skip the prefix.

Let the user input ends with the string bye. The input only contains English letters and spaces, where each word is separated by spaces.

For example the user input the following:

```
appLicaTion   Zoo   a    banana
batman  an  baMboO   battle    Applying
apply       boTTle                 ByE
```

Then your program should output:

```
a
 n
 pplication
    y
     ing
bamboo
  nana
  tman
   tle
 ottle
zoo
```

Question 2

We want to find the minimal number of steps we need to modify a string `str1` into another string `str2`. In each step, we can perform one of the following operations:

- Insert one character at an arbitrary position of the string.
- Delete one character at an arbitrary position of the string.
- Substitute one character at an arbitrary position of the string into another character.

As the above operations are symmetric, you can perform them on either `str1` or `str2`. In this question, we can assume that the strings `str1` and `str2` have lengths at most 10 (excluding the null character \0).

Task: Read `str1` and `str2` (may have spaces in each string) from user input, and print the minimal number of steps to modify `str1` into `str2`.

Sample Input 1	Example 2
`kitten` `sitting`	`trolley` `tolling`
Sample Output 1	Sample Output 2
3	4
Reason	Reason
`**k**itten` → `**s**itten` (Substitution) `sitte**n**` → `sitti**n**` (Substitution) `sittin` → `sitting**g**` (Insertion)	`**tr**olley` → `tolley` (Deletion) `toll**e**y` → `toll**i**y` (Substitution) `tolli**y**` → `tolli**n**` (Substitution) `tollin` → `tolling**g**` (Insertion)

Hint:

Let `str1` be $a_1 a_2 \ldots a_m$ and `str2` be $b_1 b_2 \ldots b_n$, where each a_k and b_k is a character. Let $d(i, j)$ be the minimal number of steps to modify the substring $a_1 a_2 \ldots a_i$ into the substring $b_1 b_2 \ldots b_j$.

- Express $d(i, j)$ in terms of $d(i, j-1)$, $d(i-1, j)$ and $d(i-1, j-1)$ under different cases.
    - If $a_i \neq b_j$, perform one of the 3 operations that results in the minimal number of steps.
- Consider the boundary cases $d(i, 0)$ and $d(0, j)$ separately.

Question 3

Given a 11×11 `char` array that represents a `maze`.

- The boundaries of the maze, i.e., `maze[0][i]`, `maze[i][0]`, `maze[10][i]` and `maze[i][10]` for all `i` = `0`, `1`, …, `10`, are walls.
- The wall is represented by the character `#`, and the passage is represented by spaces.
- The starting point of the maze is at maze[1][1], and the exit of the maze is at maze[9][9].
- In every step, you can move to the passage either on the top, bottom, left, or right only.
- You cannot pass through walls or break walls.

Task: Read the maze from user input. Print 1 if there is a path from the “starting point” to the exit, or print 0 if there is not.

Sample input 1	Sample input 2
``` ########### # # # # #### # ## # # # ### ### # # ## ### # ### ### # # # ### # # ### # # # # # # ########### ```	``` ########### # # # # ###### ## # # # ####### # # ## ### # ### ### # # # ### # # ### # # # # # # # ########### ```
Sample output 1	Sample output 2
1	0
Reason	Reason
``` ########### # # # # #### # ## # # # ### ### # # ## ### # ### ### # # # ### # # ### # # # # # # ########### ```	The starting point and the exit are isolated.

Question 4

Suppose you have the address of a member in a structure, how can you access other members in the same structure? This is somehow useful when you write advanced C codes, like the drivers.

Now, you are given the following incomplete program.

```
#include <stdio.h>

struct foo {
    char c1;
    int i;
    char c2;
    double d;
};

void func(double *d)
{
    // print the integer i in the structure
}

int main(void){
    struct foo bar;
    bar.i = 10; // for example
    func(&bar.d);
    return 0;
}
```

Your task is to fill in the function func() such that it will print the integer i in the structure. In the above example, your program should output 10.

Hint: Try to find the starting address of the structure from the given address.